

LSTM (Long Short-Term Memory)

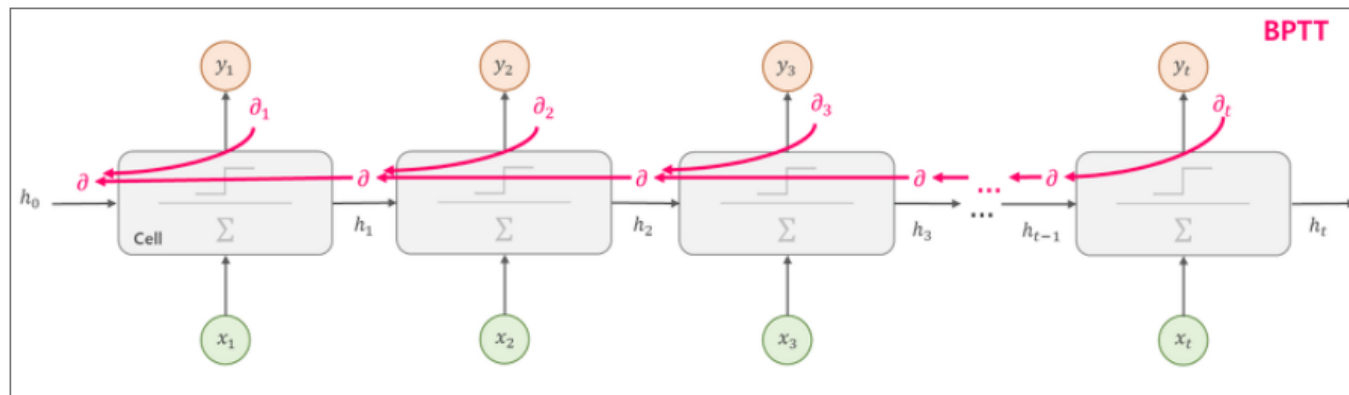
Fall 2025

School of IT Convergence

Prof. Dongsup Jin

RNN Problem

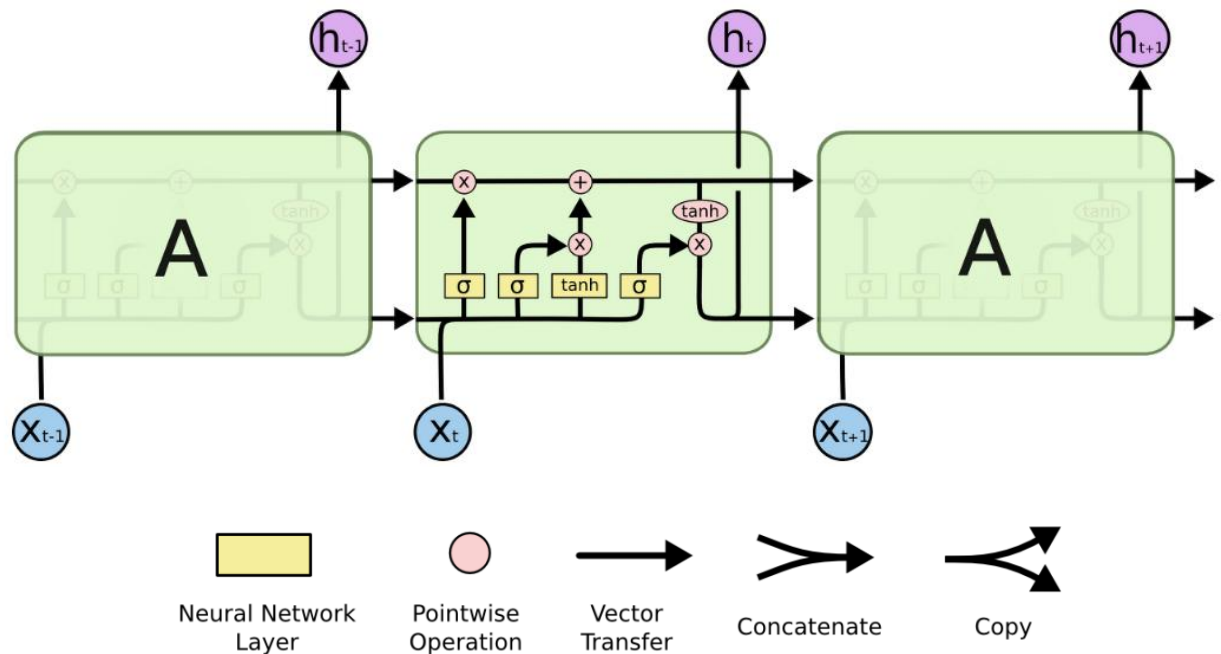
- RNN (Recurrent Neural Network) Problem
 - BPTT (Back-Propagation Through Time)
 - Vanishing Gradient Problem



1. LSTM (Long Short-Term Memory)
2. GRU (Gated Recurrent Unit)

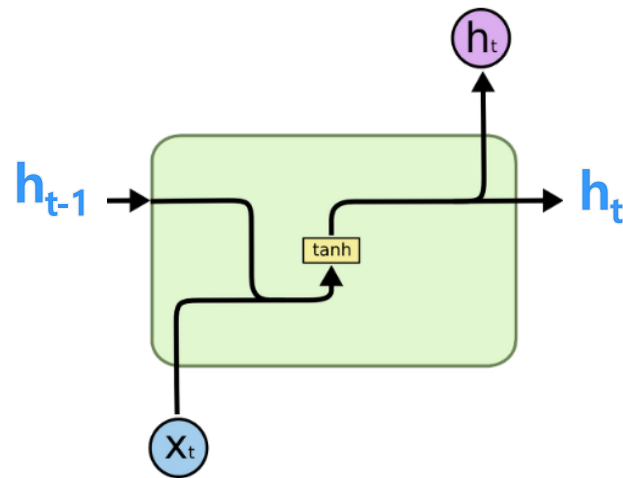
LSTM (Long Short-Term Memory) (1)

- LSTM (Long Short-Term Memory)
 - A structure to alleviate the vanishing gradient problem
 - Add one state and three new gates to each neuron in the hidden layer
 - Cell state
 - Forget gate
 - Input gate
 - Output gate

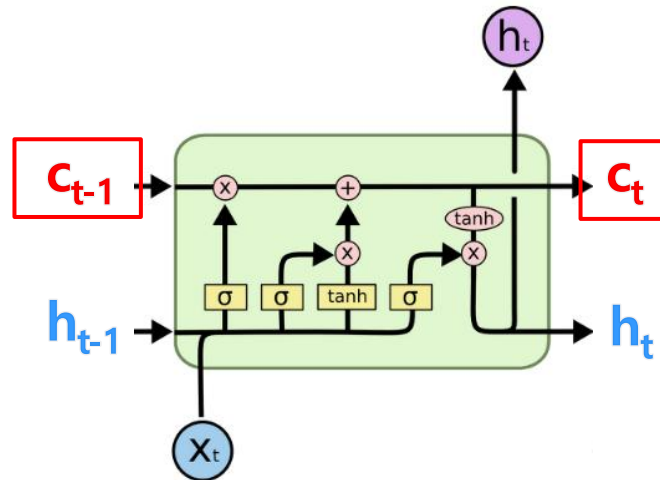


LSTM (Long Short-Term Memory) (2)

- Cell state
 - Called by 'memory cell'
 - Memory mechanism
 - Assume that all necessary information from the past to time t is stored
 - Output the hidden state to the output (next) layer, based on this memory



RNN hidden node



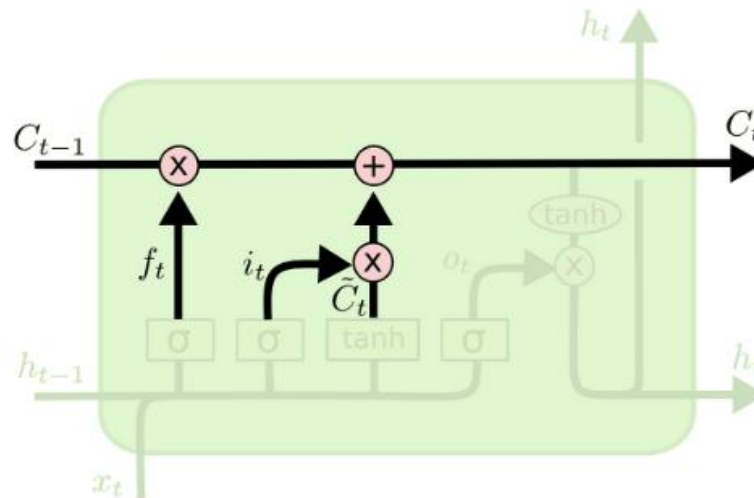
LSTM hidden node

LSTM (Long Short-Term Memory) (3)

- Cell state

- f_t : forget gate
- C_{t-1} : cell state in the t-1
- i_t : input gate
- $\tilde{C}_t$: a vector in the middle

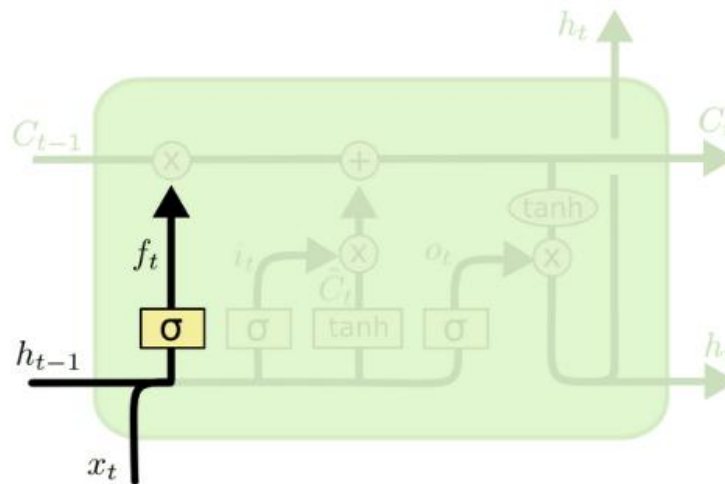
$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$



Cell state update

LSTM (Long Short-Term Memory) (4)

- Forget gate
 - Decide which information to discard
 - Determined by the sigmoid layer
 - Structure
 - Input: h_{t-1} and x_t
 - Output : f_t (0 ~ 1 value, 0: drop all and 1: preserve all)



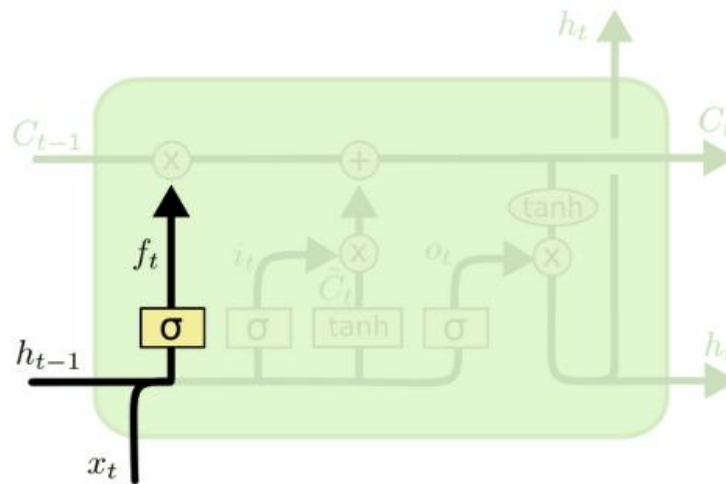
forget gate

LSTM (Long Short-Term Memory) (5)

- Forget gate

- h_{t-1} : hidden vector in the t-1
- x_t : input vector
- W_f : **weight**
- b_f : **bias**
- σ : sigmoid function

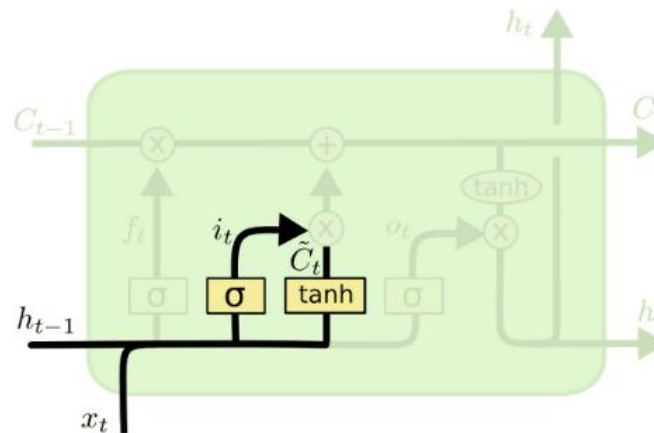
$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$



forget gate

LSTM (Long Short-Term Memory) (6)

- Input gate
 - Decide which information is to be stored in the cell state
 - Decide which value to update by the sigmoid layer
 - Create a new vector by the tangent hyperbolic layer
 - Structure
 - Input: h_{t-1} and x_t
 - Output: $i_t * \tilde{C}_t$



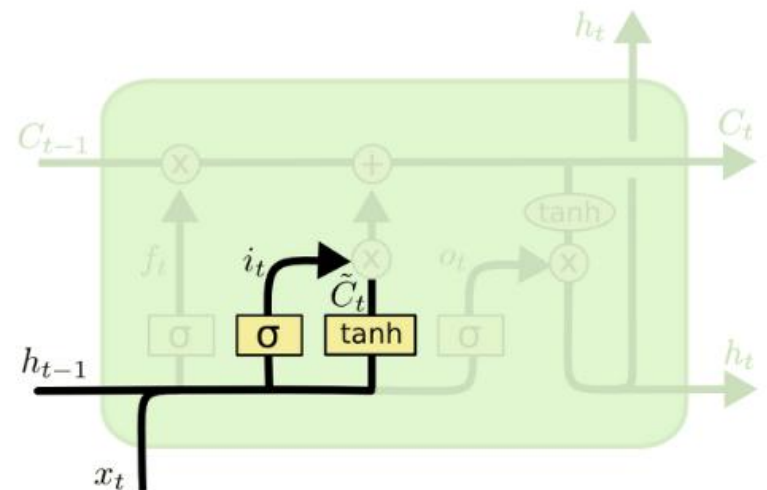
input gate

LSTM (Long Short-Term Memory) (7)

- Input gate

- h_{t-1} : hidden vector in the t-1
- x_t : input vector
- W_i : **weight**
- b_i : **bias**
- σ : sigmoid function
- W_C : weight
- b_C : bias
- $\tanh$: tangent hyperbolic function

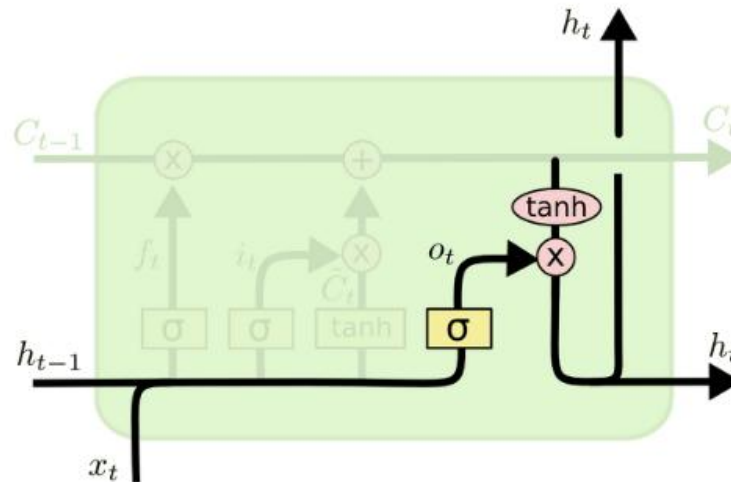
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$



input gate

LSTM (Long Short-Term Memory) (8)

- Output gate
 - Decide what to output
 - Structure
 - Input: h_{t-1} and x_t
 - Output: o_t



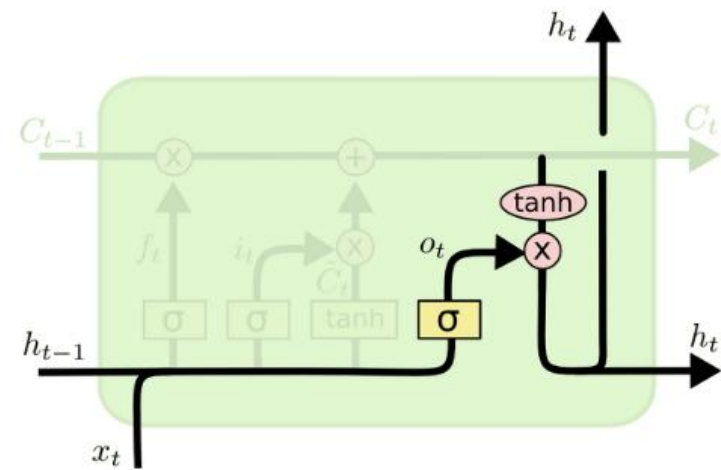
output gate

LSTM (Long Short-Term Memory) (9)

- Output gate

- h_{t-1} : hidden vector in the t-1
- x_t : input vector
- W_o : **weight**
- b_o : **bias**
- σ : sigmoid function
- C_t : cell state
- $\tanh$: tangent hyperbolic function

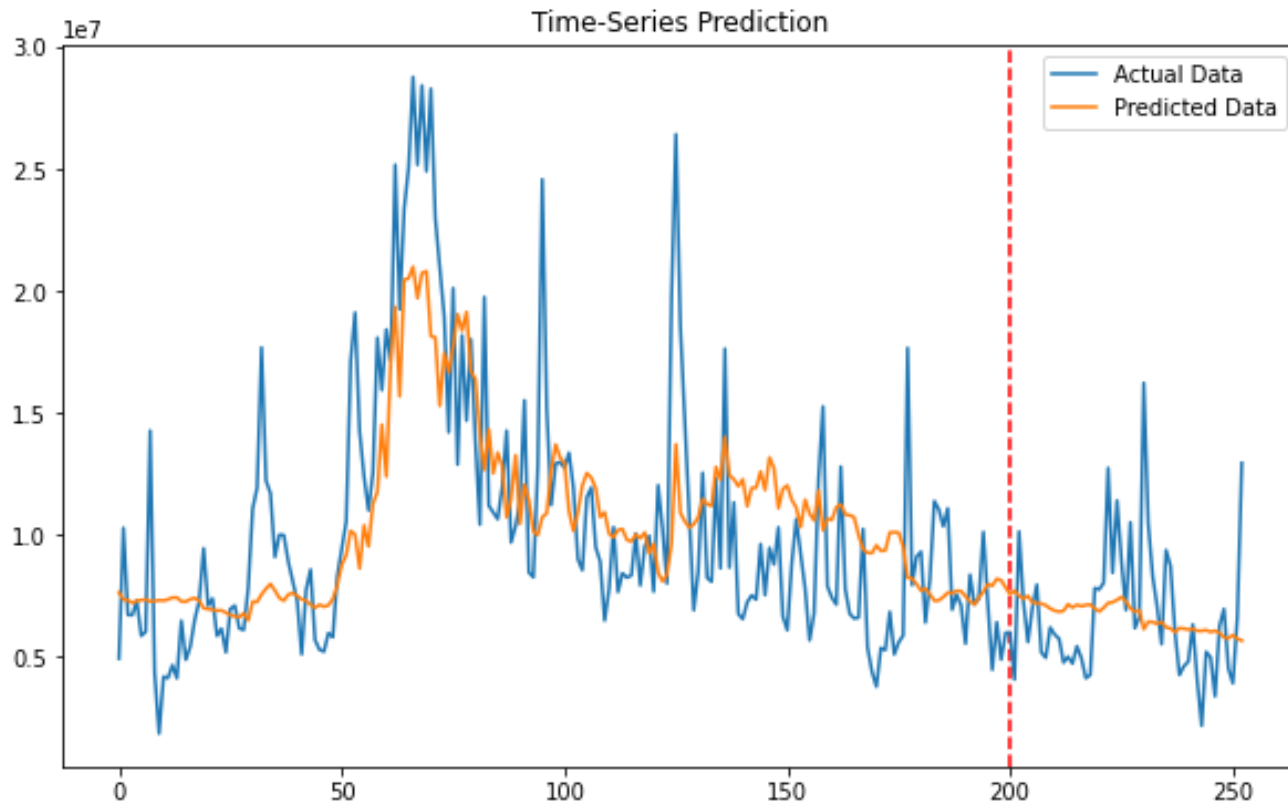
$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$
$$h_t = o_t * \tanh(C_t)$$



output gate

LSTM in Pytorch (1) – starbucks stock prediction

- (Example) Starbucks stock prediction



LSTM in Pytorch (2) – starbucks stock prediction

1. Import several packages and modules

```
import os
import time

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader
from torch.autograd import Variable

from tqdm import tqdm_notebook
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.model_selection import train_test_split
```

2. Set hyper parameters

```
device = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')
num_epochs = 1000
learning_rate = 0.0001

input_size = 5
hidden_size = 2
num_layers = 1
num_classes = 1
```

LSTM in Pytorch (3) – starbucks stock prediction

3. Load dataset

```
data=pd.read_csv('../chap07/data/SBUX.csv')
```

```
data['Date'] = pd.to_datetime(data['Date'])  
data.set_index('Date', inplace=True)
```

```
data['Volume'] = data['Volume'].astype(float)
```

```
X=data.iloc[:, :-1]  
y=data.iloc[:, 5:6]
```

```
ms = MinMaxScaler()  
ss = StandardScaler()
```

```
X_ss = ss.fit_transform(X)  
y_ms = ms.fit_transform(y)
```

```
X_train = X_ss[:200, :]  
X_test = X_ss[200:, :]
```

```
y_train = y_ms[:200, :]  
y_test = y_ms[200:, :]
```

```
X_train_tensors = Variable(torch.Tensor(X_train))  
X_test_tensors = Variable(torch.Tensor(X_test))
```

```
y_train_tensors = Variable(torch.Tensor(y_train))  
y_test_tensors = Variable(torch.Tensor(y_test))
```

```
X_train_tensors_f = torch.reshape(X_train_tensors, (X_train_tensors.shape[0], 1, X_train_tensors.shape[1]))  
X_test_tensors_f = torch.reshape(X_test_tensors, (X_test_tensors.shape[0], 1, X_test_tensors.shape[1]))
```

```
print(data.dtypes)
```

```
Date      object  
Open      float64  
High      float64  
Low       float64  
Close     float64  
Adj Close float64  
Volume    int64  
dtype: object
```

```
print(X)  
print(y)
```

	Open	High	Low	Close	Adj Close
Date					
2019-12-11	86.260002	86.870003	85.849998	86.589996	84.145752
2019-12-12	88.000000	88.889999	87.540001	88.209999	85.720032
2019-12-13	88.019997	88.790001	87.580002	88.669998	86.167046
2019-12-16	89.139999	89.300003	88.430000	88.779999	86.273941
2019-12-17	88.870003	88.970001	87.470001	88.129997	85.642288
...	...	...	...	...	...
2020-12-04	101.349998	102.940002	101.070000	102.279999	101.442787
2020-12-07	102.010002	102.220001	100.690002	101.410004	100.579918
2020-12-08	100.370003	101.570000	100.010002	101.209999	100.381554
2020-12-09	101.940002	102.209999	100.099998	100.400002	99.578186
2020-12-10	103.510002	106.089996	102.750000	105.389999	104.527336
[253 rows x 5 columns]					
Volume					
Date					
2019-12-11	4921900.0				
2019-12-12	10282100.0				
2019-12-13	67114100.0				
2019-12-16	6705600.0				
2019-12-17	7296900.0				
...	...				
2020-12-04	6952700.0				
2020-12-07	4514800.0				
2020-12-08	3911300.0				
2020-12-09	6629900.0				
2020-12-10	12939200.0				

```
print("Training Shape", X_train.shape, y_train.shape)  
print("Testing Shape", X_test.shape, y_test.shape)
```

```
Training Shape (200, 5) (200, 1)  
Testing Shape (53, 5) (53, 1)
```

```
print("Training Shape", X_train_tensors_f.shape, y_train_tensors.shape)  
print("Testing Shape", X_test_tensors_f.shape, y_test_tensors.shape)
```

```
Training Shape torch.Size([200, 1, 5]) torch.Size([200, 1])  
Testing Shape torch.Size([53, 1, 5]) torch.Size([53, 1])
```

LSTM in Pytorch (4) – starbucks stock prediction

4. Define LSTM model

```
class LSTM(nn.Module):
    def __init__(self, num_classes, input_size, hidden_size, num_layers, seq_length):
        super(LSTM, self).__init__()
        self.num_classes = num_classes
        self.num_layers = num_layers
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.seq_length = seq_length

        self.lstm = nn.LSTM(input_size=input_size, hidden_size=hidden_size,
                             num_layers=num_layers, batch_first=True)
        self.fc_1 = nn.Linear(hidden_size, 128)
        self.fc = nn.Linear(128, num_classes)

        self.relu = nn.ReLU()

    def forward(self, x):
        h_0 = Variable(torch.zeros(self.num_layers, x.size(0), self.hidden_size))
        c_0 = Variable(torch.zeros(self.num_layers, x.size(0), self.hidden_size))

        output, (hn, cn) = self.lstm(x, (h_0, c_0))
        hn = hn.view(-1, self.hidden_size)
        out = self.relu(hn)
        out = self.fc_1(out)
        out = self.relu(out)
        out = self.fc(out)
        return out
```

LSTM in Pytorch (5) – starbucks stock prediction

5. Train module

```
for epoch in range(num_epochs):
    outputs = model.forward(X_train_tensors_f)
    optimizer.zero_grad()
    loss = criterion(outputs, y_train_tensors)
    loss.backward()
    optimizer.step()
    if epoch % 100 == 0:
        print("Epoch: %d, loss: %1.5f" % (epoch, loss.item()))
```



```
Epoch: 0, loss: 0.10137
Epoch: 100, loss: 0.02986
Epoch: 200, loss: 0.02634
Epoch: 300, loss: 0.02480
Epoch: 400, loss: 0.02357
Epoch: 500, loss: 0.02255
Epoch: 600, loss: 0.02166
Epoch: 700, loss: 0.02087
Epoch: 800, loss: 0.02018
Epoch: 900, loss: 0.01957
```

6. Predict and plot

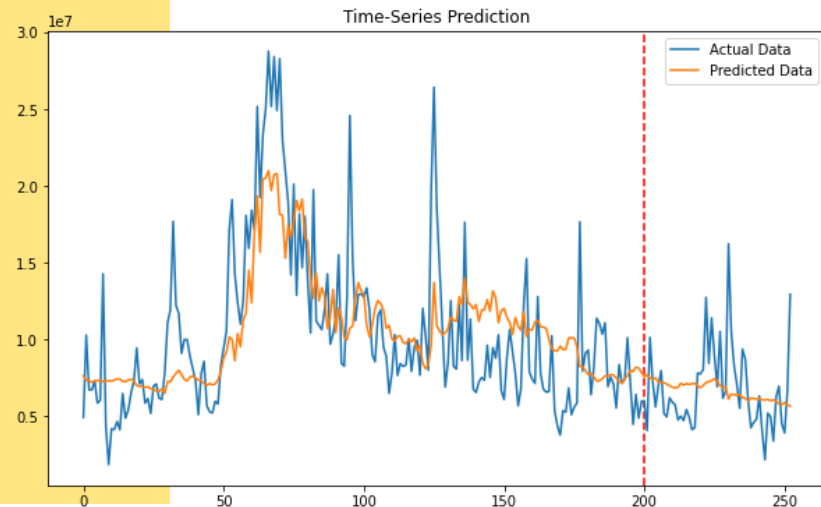
```
df_x_ss = ss.transform(data.iloc[:, :-1])
df_y_ms = ms.transform(data.iloc[:, -1:])

df_x_ss = Variable(torch.Tensor(df_x_ss))
df_y_ms = Variable(torch.Tensor(df_y_ms))
df_x_ss = torch.reshape(df_x_ss, (df_x_ss.shape[0], 1, df_x_ss.shape[1]))

train_predict = model(df_x_ss)
predicted = train_predict.data.numpy()
label_y = df_y_ms.data.numpy()

predicted = ms.inverse_transform(predicted)
label_y = ms.inverse_transform(label_y)
plt.figure(figsize=(10,6))
plt.axvline(x=200, c='r', linestyle='--')

plt.plot(label_y, label='Actual Data')
plt.plot(predicted, label='Predicted Data')
plt.title('Time-Series Prediction')
plt.legend()
plt.show()
```



LSTM Example: Stock prediction (1)

- (Example) Stock Prediction



LSTM Example: Stock prediction (2)

- (Example) Stock Prediction



LSTM Example: Stock prediction (3)

- (Example) Stock Prediction



Q & A
